Projects assignments descriptions

Bioinspired Algorithms for Optimization

This document contains the project assignment that each group must complete for the practical component of the subject. Please note that the group you select will determine which project your team must carry out.

General Instructions for the assignment

Objectives of the project assignment

The aim of this project is for students to analyze, design and implement solutions to an optimization problem using the techniques and mechanisms of bioinspired algorithms taught in the **Bioinspired Algorithms for Optimization (BAO)** course.

To do so, students will develop a programming project in Python language in a group using the PyCharm programming environment, which provides the necessary tools for the development, testing, documentation and debugging of source code in Python. Other environments can be used but must be consulted first with the professors.

Instructions for the development of the Project

Each student must select one of the available problems. The problem will be solved using the techniques learned during the lectures of the course. The solving must include:

- Solving it with at least one evolutionary algorithm and one swarm intelligence algorithm.
- Using different representations for the problem, comparing and discussing which one is the better.
- If the problem has constraints, testing different constraint handling techniques.
- If the problem is multi-objective, testing different multi-objective algorithms and metrics.
- Fine-tuning to find the best hyperparameter configuration for each algorithm used, showing the results of these experiments with all the configurations tested in the report.
- The evaluation of the problem must be done taking into account that these methods are stochastic, so more than 30 executions for each algorithm configuration must be performed for the sake of a statistically significant comparison.
- The evaluation of the problem must include comparisons in terms of quality and speed (runtime), as well as showing for the most significant configurations the convergence and diversity during the iterative process using some graphics.
- For the comparison of two algorithmic configurations, statistical significance testing must be performed.
- Using advanced techniques (e.g. parallelism, co-evolution, memetic algorithms, ...) is allowed, but not mandatory.

Apart from developing a Python project to solve the problem, the students will have to write a report following the scheme provided in the file "Report template.pdf", explaining the conceptualization, analysis, design, implementation and experimental results obtained during the project development. Also, a 10 minutes presentation explaining all of these parts must be developed and presented during the presentation sessions.

Regulations and evaluation

The project assignment should be carried out taking into account the following rules:

- The project assignment will be carried out in groups. Each group must independently develop its own project assignment and submit its own project.
- Plagiarism of the project assignment is strictly forbidden. The detection of plagiarism will result in a grade of 0 in the course for all the students involved, as well as the possibility of opening a disciplinary academic record.
- In order to be evaluated of the project assignment, it is mandatory to submit it on time, and present it during the scheduled presentation activities.
- For the development of bioinspired algorithm functionalities needed in the project assignment, student must use the library **inspyred**. The use of other bioinspired libraries is, in principle, not allowed. If you need to use other bioinspired library for your project, you must write an email to both professors (cristian.ramirez@upm.es and angel.panizo@upm.es) to request an academic tutorial appointment to discuss the need for using it.
- The use of any auxiliary library, such as numpy or pandas, is allowed.

Available problems

Group 1: Multimodal 2D bin packaging problem

Problem description

The bin packing problem is a fundamental optimization problem in which you have a set of rectangular boxes, each with specific dimensions (width and height) and a weight, along with a fixed-size rectangular bin. The objective is to fit as many boxes as possible into the bin without exceeding its maximum weight capacity.

A solution to this problem is the location (x,y) of each box and the angle of rotation (degrees) that goes inside the bin. Boxes that are excluded must be marked somehow. For a solution to be valid, boxes cannot occupy the same space, cannot go outside the bin, and the weight of all the boxes must be less than the maximum capacity of the bin.

Variation

In this version of the problem the students must implement **multimodal** algorithms, therefore, we are not just looking for the best solutions in this version, we want 3 different alternatives for each possible problem. Variety in the solutions is encouraged.

Data

This link (https://github.com/Oscar-Oliveira/OR-Datasets/tree/master/Cutting-and-Packing/2D/Datasets/WVINT) contains 72 instances of the Bin Packing problem, use them for the experimentation.

Group 2: Single-Objective 2D bin packaging problem with separation

Problem description

The bin packing problem is a fundamental optimization problem in which you have a set of rectangular boxes, each with specific dimensions (width and height) and a weight, along with a fixed-size rectangular bin. The objective is to fit as many boxes as possible into the bin without exceeding its maximum weight capacity.

A solution to this problem is the location (x,y) of each box and the angle of rotation (degrees) that goes inside the bin. Boxes that are excluded must be marked somehow. For a solution to be valid, boxes cannot occupy the same space, cannot go outside the bin, and the weight of all the boxes must be less than the maximum capacity of the bin.

Variation

In this version of the problem the students must implement a special fitness function that tries to fit as many boxes as possible in a bin and forces the boxes to be separated as much as possible. Below you can see an example of how this fitness function should work.

Data

This link (https://github.com/Oscar-Oliveira/OR-Datasets/tree/master/Cutting-and-Packing/2D/Datasets/WVINT) contains 72 instances of the Bin Packing problem, use them for the experimentation.

Group 3: Multi-Objective bin packing problem

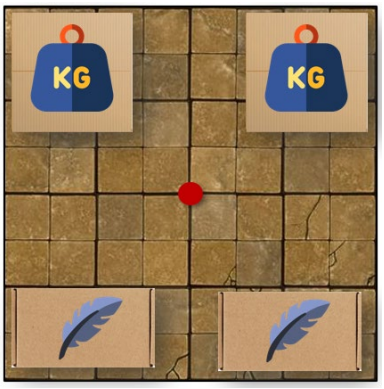
Problem description

The bin packing problem is a fundamental optimization problem in which you have a set of rectangular boxes, each with specific dimensions (width and height) and a weight, along with a fixed-size rectangular bin. The objective is to fit as many boxes as possible into the bin without exceeding its maximum weight capacity.

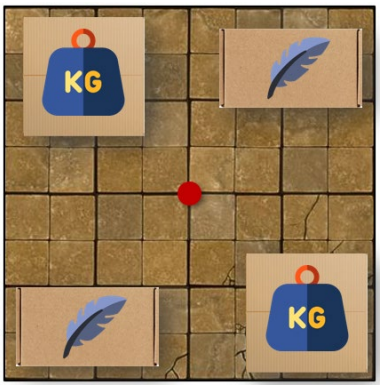
A solution to this problem is the location (x,y) of each box and the angle of rotation (degrees) that goes inside the bin. Boxes that are excluded must be marked somehow. For a solution to be valid, boxes cannot occupy the same space, cannot go outside the bin, and the weight of all the boxes must be less than the maximum capacity of the bin.

Variation

In this version of the problem, students must implement a multi-objective bin packing problem. One of the objectives must be the number of boxes fitted into the bin, as usual, and the new objective must be the balance of the boxes inside the bin. This second objective must measure how the weight is distributed within the bin. The closer the centre of mass of all the boxes is to the actual centre of the bin, the more balanced the solution. Below are two examples: one of an imbalanced solution and one of a balanced solution.



Inbalanced



Balanced

To measure how imbalanced a solution is, we assume that the weight inside each box is evenly distributed. We then calculate the centre of mass of all the boxes and measure its distance to the centre of the bin. The smaller this distance, the more balanced the solution. You can use the following formula to calculate this:

$$x_{mass} = \sum_{i \in Boxes} W_i * x_i \Big/ \sum_{i \in Boxes} W_i$$

$$y_{mass} = \sum_{i \in Boxes} W_i * y_i \Big/ \sum_{i \in Boxes} W_i$$

$$balance = \sqrt[2]{(x_{mass} - x_{centre})^2 + (y_{mass} - y_{centre})^2}$$

- W_i weight of box i
- x_i is the x component of the center of box i
- y_i is the y component of the center of box i
- x_{center} is the x component of the center of the bin
- y_{center} is the y component of the center of the bin

Data

This link (https://github.com/Oscar-Oliveira/OR-Datasets/tree/master/Cutting-and-Packing/2D/Datasets/WVINT) contains 72 instances of the Bin Packing problem, use them for the experimentation.

Group 4: Multi-Objective Course Scheduling Problem

Problem Description

The problem consists of the following: a set of events that are to be scheduled into 45 timeslots (5 days of 9 hours each); a set of rooms in which events take place; a set of students who attend the events; and a set of features satisfied by rooms and required by events. Each student attends a number of events and each room has a size.

In addition to this, some events can only be assigned to certain timeslots. Also, some events will be required to occur before other events in the timetable.

A feasible timetable is therefore one in which events have been assigned a timeslot and a room so that the following hard constraints are satisfied:

1. No student attends more than one event at the same time;
2. The room is big enough for all the attending students and satisfies all the features required by the event;
3. Only one event is put into each room in any timeslot;
4. Events are only assigned to timeslots that are pre-defined as available for those events;
5. Where specified, events are scheduled to occur in the correct order in the week

You may not always be able to schedule all of the events into the timetable in the given time without breaking some of the hard constraints. It will therefore be necessary for entrants to not place some events in the timetable (or remove them later) in order to ensure that no hard constraints are being violated. These events will then be considered unplaced.

Variation

In this version of the problem students must implement a multi-objective version of this problem. The first objective will be the room utilization, the percentage of students assisting to an event divided by the room capacity. A value of 1.0 means that the room is used at its full capacity and values near zeros means that a lot of room is being mismanaged. This objective must be maximized. The second objective will be the student schedule quality, a student schedule is considered perfect if three things happens: students do not have a day with only one class; a student has consecutive classes with no gaps; however, a student will have a gap if there are more than 3 consecutive classes.

Data

Data can be downloaded from this webpage:
https://www.eeecs.qub.ac.uk/itc2007/Login/SecretPage.php

The dataset are the ones in section *"Post Enrolment based Course Timetabling"*

Information about the data format is available here:
https://www.eeecs.qub.ac.uk/itc2007/postenrolcourse/course_post_index_files/Inputformat.htm

Extra information about the problem is available here:
https://www.eeecs.qub.ac.uk/itc2007/postenrolcourse/report/Post%20Enrolment%20based%20CourseTimetabling.pdf

Group 5: Multi-modal Course Scheduling Problem

Problem Description

The problem consists of the following: a set of events that are to be scheduled into 45 timeslots (5 days of 9 hours each); a set of rooms in which events take place; a set of students who attend the events; and a set of features satisfied by rooms and required by events. Each student attends a number of events and each room has a size.

In addition to this, some events can only be assigned to certain timeslots. Also, some events will be required to occur before other events in the timetable.

A feasible timetable is therefore one in which events have been assigned a timeslot and a room so that the following hard constraints are satisfied:

1. No student attends more than one event at the same time;
2. The room is big enough for all the attending students and satisfies all the features required by the event;
3. Only one event is put into each room in any timeslot;
4. Events are only assigned to timeslots that are pre-defined as available for those events;
5. Where specified, events are scheduled to occur in the correct order in the week

You may not always be able to schedule all of the events into the timetable in the given time without breaking some of the hard constraints. It will therefore be necessary for entrants to not place some events in the timetable (or remove them later) in order to ensure that no hard constraints are being violated. These events will then be considered unplaced.

Variation

In this version of the problem students must implement a multi-modal version of this problem. The algorithm should return, when possible, 3 different feasible solutions to the problem, variety in the solutions is encouraged.

Data

Data can be downloaded from this webpage:
https://www.eeecs.qub.ac.uk/itc2007/Login/SecretPage.php

The dataset are the ones in section *"Post Enrolment based Course Timetabling"*

Information about the data format is available here:
https://www.eeecs.qub.ac.uk/itc2007/postenrolcourse/course_post_index_files/Inputformat.htm

Extra information about the problem is available here:
https://www.eeecs.qub.ac.uk/itc2007/postenrolcourse/report/Post%20Enrolment%20based%20CourseTimetabling.pdf

Group 6: Constrained Course Scheduling Problem

Problem Description

The problem consists of the following: a set of events that are to be scheduled into 45 timeslots (5 days of 9 hours each); a set of rooms in which events take place; a set of students who attend the events; and a set of features satisfied by rooms and required by events. Each student attends a number of events and each room has a size.

In addition to this, some events can only be assigned to certain timeslots. Also, some events will be required to occur before other events in the timetable.

A feasible timetable is therefore one in which events have been assigned a timeslot and a room so that the following hard constraints are satisfied:

1. No student attends more than one event at the same time;
2. The room is big enough for all the attending students and satisfies all the features required by the event;
3. Only one event is put into each room in any timeslot;
4. Events are only assigned to timeslots that are pre-defined as available for those events;
5. Where specified, events are scheduled to occur in the correct order in the week

You may not always be able to schedule all of the events into the timetable in the given time without breaking some of the hard constraints. It will therefore be necessary for entrants to not place some events in the timetable (or remove them later) in order to ensure that no hard constraints are being violated. These events will then be considered unplaced.

Variation

In this version of the problem students must implement a version of the course scheduling problem where student preferences are considered. In particular, a candidate timetable is penalized equally for each occurrence of the following soft constraint violations:

- A student has a class in the last slot of the day;
- Student has more than two classes consecutively;
- A student has a single class on a day.

Data

Data can be downloaded from this webpage:
https://www.eeecs.qub.ac.uk/itc2007/Login/SecretPage.php

The dataset are the ones in section *"Post Enrolment based Course Timetabling"*

Information about the data format is available here:
https://www.eeecs.qub.ac.uk/itc2007/postenrolcourse/course_post_index_files/Inputformat.htm

Extra information about the problem is available here:
https://www.eeecs.qub.ac.uk/itc2007/postenrolcourse/report/Post%20Enrolment%20based%20CourseTimetabling.pdf

Group 7: Multi-objective Capacitated Vehicle Routing Problem

Problem Description

The Capacitated Vehicle Routing Problem (CVRP) is a fundamental logistics and supply chain optimization problem in which you have a central depot, a fleet of identical delivery vehicles with a fixed carrying capacity, and a set of geographically dispersed customers, each requiring a specific demand of goods. The objective is to design a set of closed delivery routes to service all customers while minimizing the overall transportation cost.

A solution to this problem is a set of distinct routes, where each route defines the exact sequence of customers assigned to a specific vehicle. For a solution to be valid, every customer must be visited exactly once by a single vehicle, all routes must start and end at the central depot (located at 0,0), and the total demand of all customers on any given route must be less than or equal to the maximum weight/volume capacity of the vehicle.

Variation

In this version of the problem, the students must implement a multi-objective approach. The objectives to consider are:

- Minimizing total distance
- Balancing the demand covered by each vehicle, i.e. demand covered by each vehicle should be as similar as possible

Data

100 samples of data can be downloaded from this webpage:
https://github.com/PyVRP/Instances/tree/main/CVRP

Each sample in the dataset consists of a file with the format "X-n[T]-k[V].vrp", where [T] is the number of tasks and [V] is the number of available vehicles. Each file is text formatted and contains the capacity for each vehicle, the coordinates (NODE_COORD_SECTION) for the tasks and the demand (DEMAND_SECTION) of each task.

Group 8: Single-objective Capacitated Vehicle Routing Problem

Problem Description

The Capacitated Vehicle Routing Problem (CVRP) is a fundamental logistics and supply chain optimization problem in which you have a central depot, a fleet of identical delivery vehicles with a fixed carrying capacity, and a set of geographically dispersed customers, each requiring a specific demand of goods. The objective is to design a set of closed delivery routes to service all customers while minimizing the overall transportation cost.

A solution to this problem is a set of distinct routes, where each route defines the exact sequence of customers assigned to a specific vehicle. For a solution to be valid, every customer must be visited exactly once by a single vehicle, all routes must start and end at the central depot (located at 0,0), and the total demand of all customers on any given route must be less than or equal to the maximum weight/volume capacity of the vehicle.

Variation

In this version of the problem, you will minimize the total distance travelled by the fleet, and at the same time you will consider the following additional constraints:

- Capacity Constraint: The total demand of all customers on a route cannot exceed the vehicle's maximum carrying capacity.
- Maximum Route Length Constraint: To simulate driver shift limits, no single route can exceed a maximum allowable distance (e.g., for this exercise, set the maximum distance of any route to 2.5 times the distance from the depot to the farthest customer in the instance).
- Maximum Stops Constraint: A vehicle can visit a maximum of 10 customers per route to account for loading/unloading time limits. If more customers are planned for a specific vehicle, it would have to come back to depot after the 15th customer and continue its route afterwards.

Data

100 samples of data can be downloaded from this webpage:
https://github.com/PyVRP/Instances/tree/main/CVRP

Each sample in the dataset consists of a file with the format "X-n[T]-k[V].vrp", where [T] is the number of tasks and [V] is the number of available vehicles. Each file is text formatted and contains the capacity for each vehicle, the coordinates (NODE_COORD_SECTION) for the tasks and the demand (DEMAND_SECTION) of each task.

Group 9: Multi-modal Capacitated Vehicle Routing Problem

Problem Description

The Capacitated Vehicle Routing Problem (CVRP) is a fundamental logistics and supply chain optimization problem in which you have a central depot, a fleet of identical delivery vehicles with a fixed carrying capacity, and a set of geographically dispersed customers, each requiring a specific demand of goods. The objective is to design a set of closed delivery routes to service all customers while minimizing the overall transportation cost.

A solution to this problem is a set of distinct routes, where each route defines the exact sequence of customers assigned to a specific vehicle. For a solution to be valid, every customer must be visited exactly once by a single vehicle, all routes must start and end at the central depot (located at 0,0), and the total demand of all customers on any given route must be less than or equal to the maximum weight/volume capacity of the vehicle.

Variation

In this version of the problem, the students must implement multimodal algorithms. In real-world logistics, a single "optimal" mathematical route might be unworkable on a given day due to unmodeled disruptions (e.g., sudden road closures or driver preferences). Therefore, we are not just looking for the single best solution. We want 3 different, highly diverse route alternatives for each problem instance.

To be considered a valid multimodal output, all 3 alternative solutions must be "high quality" (e.g., their total distance must be within 5% of the best-found solution), but the actual assignment of customers to routes must differ significantly between the alternatives. Variety in the structural solutions is highly encouraged.

Data

100 samples of data can be downloaded from this webpage:
https://github.com/PyVRP/Instances/tree/main/CVRP

Each sample in the dataset consists of a file with the format "X-n[T]-k[V].vrp", where [T] is the number of tasks and [V] is the number of available vehicles. Each file is text formatted and contains the capacity for each vehicle, the coordinates (NODE_COORD_SECTION) for the tasks and the demand (DEMAND_SECTION) of each task.

Group 10: Single-objective Diet Optimization Problem

Problem Description

The Diet Optimization Problem is a classic selection problem where the goal is to choose a combination of food items to fulfill daily dietary needs. You have a database of foods, each with associated nutritional values (calories, proteins, carbohydrates, fats, etc.).

A solution to this problem is a set of selected food items for a weekly meal plan. In this weekly meal plan, to simplify and adapt the problem, each of the seven days of the week will be composed of five meals (breakfast, morning snack, lunch, afternoon snack and dinner). Both snacks are composed of just 1 food item that can be fruit or sugar item. Breakfast is composed of three items: one breakfast drink (milk, coffee, tea or juice) and two breakfast foods (cereals, eggs, fruits or bacon). Finally, both lunch and dinner consists of three items: one drink and two foods of any kind. For a solution to be valid, the selected foods (of their corresponding types according to the meal) must meet a basic set of minimum daily requirements to ensure the diet remains healthy.

In this problem, as the plan is to be made for a specific user, we will have a user profile where the number of calories the user must consume daily is defined. We must make sure that for each day of the week, the total number of calories consumed in the proposed solution is as close as possible to this number.

Variation

In this version of the problem, apart from optimizing the number of calories, students will try to find solutions that satisfy all of the following constraints for the specific user profile:

1. **Caloric Constraint:** The total amount of daily calories cannot be smaller than 80% of the defined number of calories and cannot be higher than 120% of this number.
2. **Carbohydrate Limit:** The percentage of daily carbohydrates (in comparison to the rest of macronutrients) must remain between 45% and 65%.
3. **Fat Limit:** The percentage of daily carbohydrates (in comparison to the rest of macronutrients) must remain between 20% and 35%.

4. **Protein Limit:** The percentage of daily carbohydrates (in comparison to the rest of macronutrients) must remain between 10% and 35%.
5. **User allergies:** No single food item can be selected if it is included in the user food allergies group.
6. **User preferences:** We must assure that food items meet user preferences, where food items in "like" groups will have a positive influence, while food items in "dislike" groups will have a negative influence.

Data

This link (https://github.com/javierquesadapajares/NutritionPlanning/blob/main/PROJECT/data/food_database_dump.sql) contains a SQL dump of a comprehensive food database with macronutrient profiles. It contains information about 5 users profiles which will be used as test subjects (our samples or datasets).

To read the database, you will need to have MySQL, MariaDB or other SQL server installed in your computer and restore the dump.sql file. Afterwards, you can use the following script to read the databases accordingly to the problem structure: https://github.com/javierquesadapajares/NutritionPlanning/blob/main/PROJECT/src/utilidades/database.py

To filter types of food and compute macronutrient proportions, you can use the following: https://github.com/javierquesadapajares/NutritionPlanning/blob/main/PROJECT/src/utilidades/funciones_auxiliares.py

IMPORTANT NOTE: The database is in Spanish, you will need to translate it.

Group 11: Multi-objective Diet Optimization Problem

Problem Description

The Diet Optimization Problem is a classic selection problem where the goal is to choose a combination of food items to fulfill daily dietary needs. You have a database of foods, each with associated nutritional values (calories, proteins, carbohydrates, fats, etc.).

A solution to this problem is a set of selected food items for a weekly meal plan. In this weekly meal plan, to simplify and adapt the problem, each of the seven days of the week will be composed of five meals (breakfast, morning snack, lunch, afternoon snack and dinner). Both snacks are composed of just 1 food item that can be fruit or sugar item. Breakfast is composed of three items: one breakfast drink (milk, coffee, tea or juice) and two breakfast foods (cereals, eggs, fruits or bacon). Finally, both lunch and dinner consists of three items: one drink and two foods of any kind. For a solution to be valid, the selected foods (of their corresponding types according to the meal) must meet a basic set of minimum daily requirements to ensure the diet remains healthy.

In this problem, as the plan is to be made for a specific user, we will have a user profile where the number of calories the user must consume daily is defined. We must make sure

that for each day of the week, the total number of calories consumed in the proposed solution is as close as possible to this number.

Variation

In this version of the problem, students must implement multi-objective algorithms that will simultaneously optimize the following objectives:

1. Adjust daily calories to the defined number of daily calories for the specific user.
2. Adjust daily carbohydrates so their percentage is 55% of macronutrients.
3. Adjust daily fats so their percentage is 27.5% of macronutrients.
4. Adjust daily proteins so their percentage is 22.5% of macronutrients.

For simplification of the problem, objectives 2, 3 and 4 can be combined into an objective called “macronutrients adjusting”.

Data

This link (https://github.com/javierquesadapajares/NutritionPlanning/blob/main/PROJECT/data/food_database_dump.sql) contains a SQL dump of a comprehensive food database with macronutrient profiles. It contains information about 5 users profiles which will be used as test subjects (our samples or datasets).

To read the database, you will need to have MySQL, MariaDB or other SQL server installed in your computer and restore the dump.sql file. Afterwards, you can use the following script to read the databases accordingly to the problem structure: https://github.com/javierquesadapajares/NutritionPlanning/blob/main/PROJECT/src/utilidades/database.py

To filter types of food and compute macronutrient proportions, you can use the following: https://github.com/javierquesadapajares/NutritionPlanning/blob/main/PROJECT/src/utilidades/funciones_auxiliares.py

IMPORTANT NOTE: The database is in Spanish, you will need to translate it.

Group 12: Continuous Diet Optimization Problem

Problem Description

The Diet Optimization Problem is a classic selection problem where the goal is to choose a combination of food items to fulfill daily dietary needs. You have a database of foods, each with associated nutritional values (calories, proteins, carbohydrates, fats, etc.).

A solution to this problem is a set of selected food items for a weekly meal plan. In this weekly meal plan, to simplify and adapt the problem, each of the seven days of the week will be composed of five meals (breakfast, morning snack, lunch, afternoon snack and dinner). Both snacks are composed of just 1 food item that can be fruit or sugar item. Breakfast is composed of three items: one breakfast drink (milk, coffee, tea or juice) and two breakfast foods (cereals, eggs, fruits or bacon). Finally, both lunch and dinner consists

of three items: one drink and two foods of any kind. For a solution to be valid, the selected foods (of their corresponding types according to the meal) must meet a basic set of minimum daily requirements to ensure the diet remains healthy.

In this problem, as the plan is to be made for a specific user, we will have a user profile where the number of calories the user must consume daily is defined. We must make sure that for each day of the week, the total number of calories consumed in the proposed solution is as close as possible to this number.

Variation

Unlike standard diet problems that select discrete servings (e.g. 100 g of rice), real-world meal prep often involves measuring exact weights. In this version of the problem, besides selecting specific foods for the weekly plan, we must also indicate the amount of that specific food. This amount will be a real value between 0.1 and 5, which will be multiplied by the selected food calories to compute its percentage (e.g. a value of 1 corresponds to the caloric value of the food item, while a value of 0.5 corresponds to half of the food item calories, and 2 corresponds to the double of calories). Students must report solutions that contain not just food selected, but also the amounts of these foods for a more realistic diet planning.

Data

This link (https://github.com/javierquesadapajares/NutritionPlanning/blob/main/PROJECT/data/food_database_dump.sql) contains a SQL dump of a comprehensive food database with macronutrient profiles. It contains information about 5 users profiles which will be used as test subjects (our samples or datasets).

To read the database, you will need to have MySQL, MariaDB or other SQL server installed in your computer and restore the dump.sql file. Afterwards, you can use the following script to read the databases accordingly to the problem structure: https://github.com/javierquesadapajares/NutritionPlanning/blob/main/PROJECT/src/utilidades/database.py

To filter types of food and compute macronutrient proportions, you can use the following: https://github.com/javierquesadapajares/NutritionPlanning/blob/main/PROJECT/src/utilidades/funciones_auxiliares.py

IMPORTANT NOTE: The database is in Spanish, you will need to translate it.